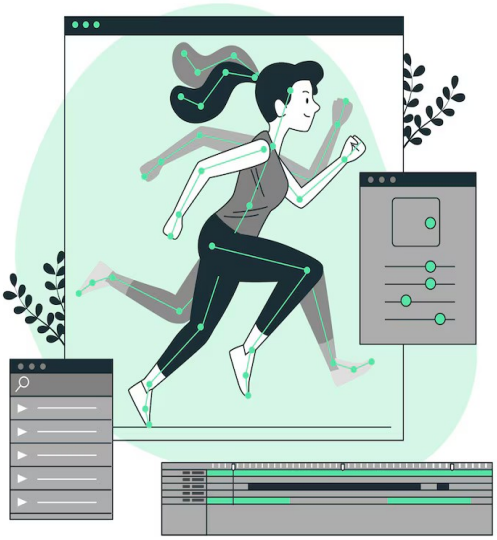
Institut Notre-Dame Beauraing

UAA6 :
Création et mise en ligne d'un site WEB

Les animations CSS

Théorie et exercices



Technique de transition
–
Option Informatique
–
Titulaire du cours : L. Embrechts

Table des matières

1 Introduction....3
2 La propriété transform....4
2.1. Définition....4
2.2. Principales fonctions de transformation....4
2.3. Exemples....4
3 La propriété transition....5
3.1. Définition....5
3.2. Syntaxe....5
3.3. Exemple....5
4 La propriété animation....6
4.1. Définition....6
4.2. Syntaxe....6
4.3. Exemple....7
5 Exercices pratiques....8
5.1. Balle qui rebondit....8
5.2. Barre de chargement....9
5.3. Let's run !....10
5.4. Danse des balles....11
5.5. Geometry Dash....12
6 CSS responsive (complément)....13
6.1. Pourquoi faire du CSS responsive ?....13
6.2. Quand faire du CSS responsive ?....13
6.3. Comment faire du CSS *responsive* ?....13
6.4. Sources....15
7 Quelques ressources pour aller plus loin....16

1 Introduction

Les interfaces web modernes se distinguent par leur dynamisme et leur interactivité. Les animations CSS permettent d'apporter du mouvement et de l'attrait visuel à une page web, tout en améliorant l'expérience utilisateur.

Ce cours présente trois propriétés essentielles pour animer des éléments HTML :

- `transition`
- `transform`
- `animation`.

Leur maîtrise est indispensable pour concevoir des sites web attractifs et professionnels

Attention : ce cours n'est pas exhaustif.

Le domaine des animations CSS est vaste et en constante évolution. L'élève est donc invité(e) à approfondir ses connaissances par des recherches personnelles, notamment en scannant les QR codes présents dans le cours qui renvoient vers des ressources complémentaires, des tutoriels ou des exemples interactifs.

2 La propriété transform



2.1. Définition

La propriété `transform` permet de modifier l'apparence d'un élément sans affecter son flux dans la page. Elle offre la possibilité d'appliquer des transformations telles que la translation, la rotation, l'agrandissement ou la réduction, et l'inclinaison.

2.2. Principales fonctions de transformation

Principales fonctions de transformation :

- `scale(<x>,<y>)` : agrandit ou réduit l'élément selon les axes X et Y
- `rotate(<angle>)` : fait pivoter l'élément selon un angle donné
- `translate(<x>,<y>)` : déplace l'élément horizontalement et/ou verticalement
- `skew(<x>,<y>)` : incline l'élément selon les axes X et Y

2.3. Exemples

```css
/* Agrandir un élément */
.grand {
  transform: scale(1.5);
}

/* Tourner un élément de 45 degrés */
.tourne {
  transform: rotate(45deg);
}

/* Déplacer un élément de 50px vers la droite et 20px vers le bas
*/
.deplace {
  transform: translate(50px, 20px);
}
```

3 La propriété `transition`



3.1. Définition

La propriété `transition` permet d'animer en douceur le passage d'une valeur à une autre pour une ou plusieurs propriétés CSS. Elle est principalement utilisée pour accompagner des changements d'état, comme un survol de souris ou un clic, afin d'éviter des modifications brutales.

3.2. Syntaxe

```
element {
  transition: <propriété> <durée> <timing-function> <délai>;
}
```

- **propriété** : la ou les propriétés CSS à animer (par exemple, `background-color`, `width`, etc.)
- **durée** : le temps que dure la transition (ex : `0.5s` pour une demi-seconde)
- **timing-function** *(optionnel)* : la courbe de vitesse de l'animation (`ease`, `linear`, `ease-in`, etc.)
- **délai** *(optionnel)* : le délai avant le début de la transition

3.3. Exemple

```css
.bouton {
  background-color: #3498db;
  color: #fff;
  transition: background-color 0.5s, transform 0.3s;
}
.bouton:hover {
  background-color: #e74c3c;
  transform: scale(1.1);
}
```

Au survol, le bouton change de couleur et grandit progressivement.

4 La propriété `animation`



4.1. Définition

La propriété `animation` permet de créer des animations complexes, composées de plusieurs étapes, qui ne dépendent pas nécessairement d'une interaction utilisateur.

Elle s'appuie sur la règle `@keyframes` pour décrire l'évolution d'un ou plusieurs styles dans le temps.

4.2. Syntaxe

```
@keyframes nom-animation {
  0%   { /* état initial */ }
  50%  { /* état intermédiaire */ }
  100% { /* état final */ }
}

.element {
  animation-name: <nom>;
  animation-duration : <durée>;
  animation-timing-function:<timing-function>;
  animation-iteration-count : <nombre-repetitions>;
  animation-direction : <sens>;
}
```

La propriété raccourcie `animation` en CSS peut être décomposée en six sous-propriétés principales, qui permettent de contrôler précisément le comportement d'une animation.

- **`animation-name`**
  Définit le nom de l'animation, qui doit correspondre à celui utilisé dans la règle `@keyframes`.
  Exemple : `animation-name: mon-animation;`
- **`animation-duration`**
  Indique la durée d'un cycle complet de l'animation (exprimée en secondes ou millisecondes).
  Exemple : `animation-duration: 2s;`
- **`animation-timing-function`**
  Définit la courbe de vitesse de l'animation, c'est-à-dire la façon dont la progression de l'animation s'accélère ou ralentit.
  Exemple : `animation-timing-function: ease-in-out;`

- **`animation-delay`**
  Spécifie le délai avant le démarrage de l'animation, après le chargement de l'élément.
  Exemple : `animation-delay: 1s;`
- **`animation-iteration-count`**
  Détermine le nombre de répétitions de l'animation (un nombre ou la valeur `infinite` pour une boucle sans fin).
  Exemple : `animation-iteration-count: infinite;`
- **`animation-direction`**
  Indique si l'animation doit recommencer au début à chaque cycle (`normal`), repartir en sens inverse (`reverse`), ou alterner (`alternate`).
  Exemple : `animation-direction: alternate;`

4.3. Exemple

```
@keyframes deplacement {
     0% {
         transform: translateX(0);
    }
     50% {
         transform: translateX(100px);
    }
     100% {
         transform: translateX(0);
    }
}
 .cercle {
     width: 50px;
     height: 50px;
     background: orange;
     border-radius: 50%;
     animation: deplacement 2s infinite;
}
```

Le cercle se déplace de gauche à droite, puis revient à sa position initiale, en boucle.

5 Exercices pratiques

5.1. Balle qui rebondit

Mets en place une animation CSS nommée `rebond` sur un élément ayant l'identifiant `balle`, qui se trouve à l'intérieur d'un conteneur centré nommé `myContener`.

L'élément `balle` contient le texte « Bonjour ».

Déroulement de l'animation

État initial (0%)

- La "balle" apparaît dans le coin supérieur gauche du conteneur.
- Sa taille est très petite (`width:50px; height:50px;`), et la taille du texte est nulle (`font-size: 0em`) donc aucun texte n'est visible.

Étape intermédiaire (50%)

- La "balle" se déplace légèrement vers la droite (`left: 20px`) et descend vers le bas du conteneur (`top: 400px`).
- Sa taille reste petite (`width: 50px; height: 50px;`) et le texte n'est toujours pas visible (`font-size: 0em`).

État final (100%)

- La "balle" atteint sa position finale, plus centrée dans le conteneur (`left: 100px; top: 100px;`).
- Elle grandit pour atteindre une taille bien plus grande (`width: 400px; height: 250px;`).
- Le texte à l'intérieur devient visible et grand (`font-size: 3em`).

Effets visuels supplémentaires

- La "balle" a un fond constitué d'une image (`decors.png`), arrondi (`border-radius: 25px`).
- L'animation dure 4 secondes et commence immédiatement.
- Le mouvement suit une courbe d'accélération naturelle.

5.2. Barre de chargement

Mets en place une barre de chargement animée au centre d'une page à fond dégradé, accompagnée d'un message "Loading..." qui clignote.

Apparence générale

- Le fond de la page est un dégradé radial allant du bleu foncé au bleu très sombre.
- Au centre de la page, une barre horizontale blanche, arrondie, représente la zone de chargement (`#conteneur`).
- À l'intérieur de cette barre, un rectangle coloré (`#mana`) se remplit progressivement de gauche à droite et un message "Loading..." (`#message`) s'affiche à l'intérieur de la barre.

Animation de la barre de progression (`#mana`)

État initial (0%)

La largeur du rectangle bleu-vert est de 0%. Il n'est donc pas visible.

État final (100%)

La largeur atteint 100% de la barre (elle est complètement remplie).

Effets visuels supplémentaires

- L'animation dure 5 secondes, de façon linéaire (la vitesse de remplissage est constante).
- La barre de progression se remplit progressivement de gauche à droite, en affichant un dégradé de vert à jaune.

Animation du message "Loading..." (`#message`)

État initial (0%)

Le message "Loading..." est complètement visible (opacité 1).

État final (100%)

Le message devient invisible (opacité 0).

Effets visuels supplémentaires

- Chaque cycle d'apparition/disparition dure 0,7 seconde.
- L'animation se répète automatiquement environ 7 fois (calculé par `calc(5 / 0.7)`), ce qui correspond à toute la durée du remplissage de la barre.

Animation du conteneur principal (`#conteneur`)

Le conteneur devient progressivement invisible 2 secondes après la fin du chargement.

5.3. Let's run !

Mets en place une scène de jeu animée où un personnage peut courir ou tirer, sur un fond illustré, grâce à l'utilisation de plusieurs animations CSS déclenchées par des boutons.

Apparence générale

- Fond de la page : une image de fond (`background.png`) couvre tout l'écran.
- Plateau de jeu : une deuxième image (`ground.png`) sert de sol, occupant toute la hauteur de la fenêtre.
- Personnage : un élément `#perso`, placé à l'intérieur d'un conteneur `#deplacement`, représente le personnage du jeu. Ce personnage n'est visible que par son image de fond, qui change selon l'animation en cours.
- Boutons : deux gros boutons ("Run !" et "Shoot !") sont affichés en bas de la page pour déclencher les animations.

Animation de déplacement

- Lorsqu'on clique sur "Run !", l'animation se lance
- L'animation dure 10 secondes
- Le personnage se déplace horizontalement de gauche à droite

Animation de course

- Lorsqu'on clique sur "Run !", l'animation se lance
- Sur une durée de 1 seconde, en boucle infinie, l'image de fond du personnage change rapidement parmi six images différentes (`course_01.png` à `course_06.png`), puis revient à la première.

Animation de tir

- Lorsqu'on clique sur "Shoot !", l'animation se lance
- Sur une durée de 1 seconde, en boucle infinie, l'image de fond du personnage change parmi six images différentes (`shoot_01.png` à `shoot_06.png`), puis revient à la première.

5.4. Danse des balles

Crée une animation visuelle composée de neuf cercles turquoise alignés horizontalement au centre de la page. Chaque cercle effectue un mouvement de rebond vertical avec un effet de changement de couleur, de taille et d'opacité, et chaque cercle démarre son animation avec un léger décalage par rapport au précédent (effet d'une « vague »).

Apparence initiale

- Neuf cercles turquoise de 50 pixels de diamètre sont affichés côte à côte, espacés régulièrement de 150 pixels, au centre horizontal de la page.
- Les cercles sont positionnés grâce à la propriété CSS `position: absolute` à l'intérieur d'un conteneur centré (`#wrapper`).

Animation

Début de l'animation (0%)

- Le cercle est à sa position de départ (en haut).
- Sa taille est normale (`scale(1)`).
- L'opacité est maximale (`opacity: 1`).
- Sa couleur est turquoise.

Milieu de l'animation (50%)

- Le cercle descend verticalement de 200 pixels.
- Il rétrécit (`scale(0.4)`), devenant plus petit.
- L'opacité diminue à 0.5.
- Sa couleur devient bleue.

Fin de l'animation (100%)

- Le cercle revient à sa position de départ (en haut).
- Il retrouve sa taille normale (`scale(1)`), son opacité maximale et sa couleur turquoise.

Effets visuels supplémentaires

- L'animation dure 1 seconde et se répète à l'infini pour chaque cercle.
- Chaque cercle commence son mouvement un peu après le précédent, produisant une impression de mouvement fluide et rythmique, comme une onde ou une vague qui se propage de gauche à droite.

5.5. Geometry Dash

Ce code crée une animation dans laquelle une boîte carrée, affichant une image de fond (`geom.png`), se déplace le long des bords d'un rectangle tout en effectuant une rotation complète sur elle-même.

Apparence initiale

- Un cadre rectangulaire de 700 pixels de large sur 400 pixels de haut, avec une bordure en pointillés, est centré verticalement et horizontalement dans la page.
- À l'intérieur de ce cadre, un carré de 100 pixels de côté, affichant l'image `geom.png`, est positionné dans le coin supérieur gauche.

Animation de la boîte

Début de l'animation (0%)

- Position : coin supérieur gauche du cadre
- Rotation : 0 degré

25%

- Position : coin supérieur droit du cadre
- Rotation : 90 degrés

50%

- Position : coin inférieur droit du cadre
- Rotation : 180 degrés

75%

- Position : coin inférieur gauche du cadre
- Rotation : 270 degrés

Fin de l'animation (100%)

- Retour à la position de départ
- Rotation : 360 degrés (un tour complet)

Effets visuels supplémentaires

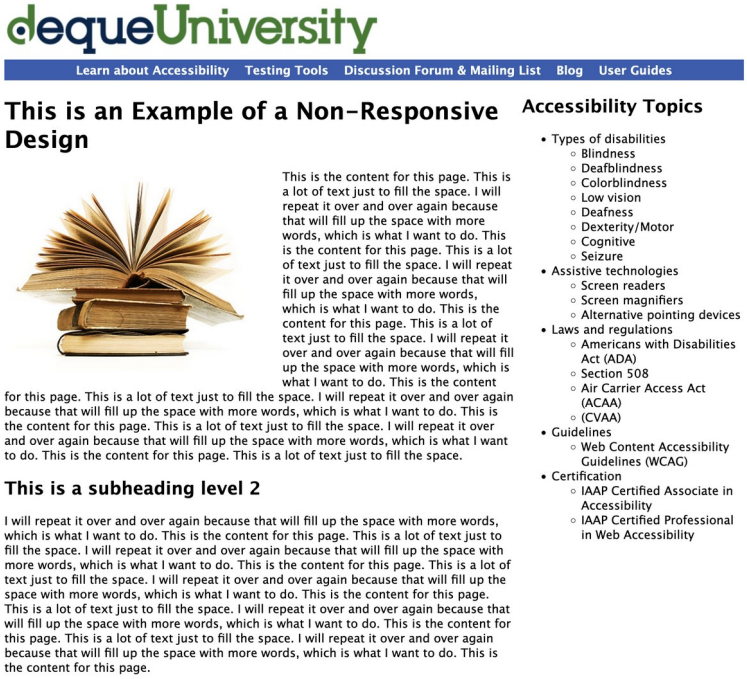
- Durée : 3 secondes pour un cycle complet.
- Répétition : L'animation se répète à l'infini, en changeant de sens à chaque boucle.

6 CSS responsive (complément)

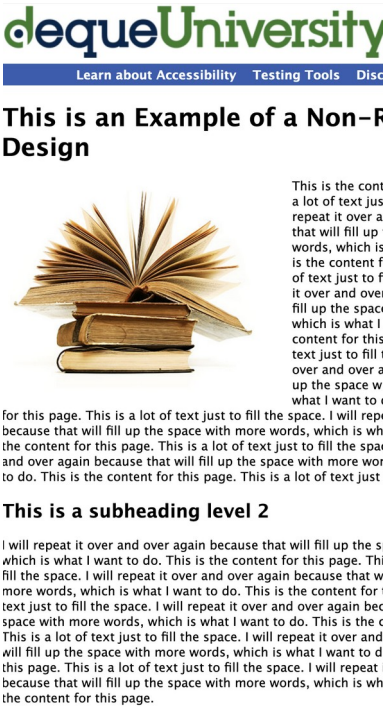
Le CSS « Responsive » consiste à appliquer des règles CSS en fonction de la taille de l'écran sur lequel s'affiche le site WEB.

6.1. Pourquoi faire du CSS responsive ?

Un site WEB peut être consulté par toutes sortes d'appareils différents : ordinateurs, tablettes, smartphones, etc. Ces appareils n'ont pas tous la même taille d'écran… et donc pas tous la même place pour accueillir les éléments du site.



Site WEB non responsive consulté sur un ordinateur



Site WEB non responsive consulté sur un smartphone

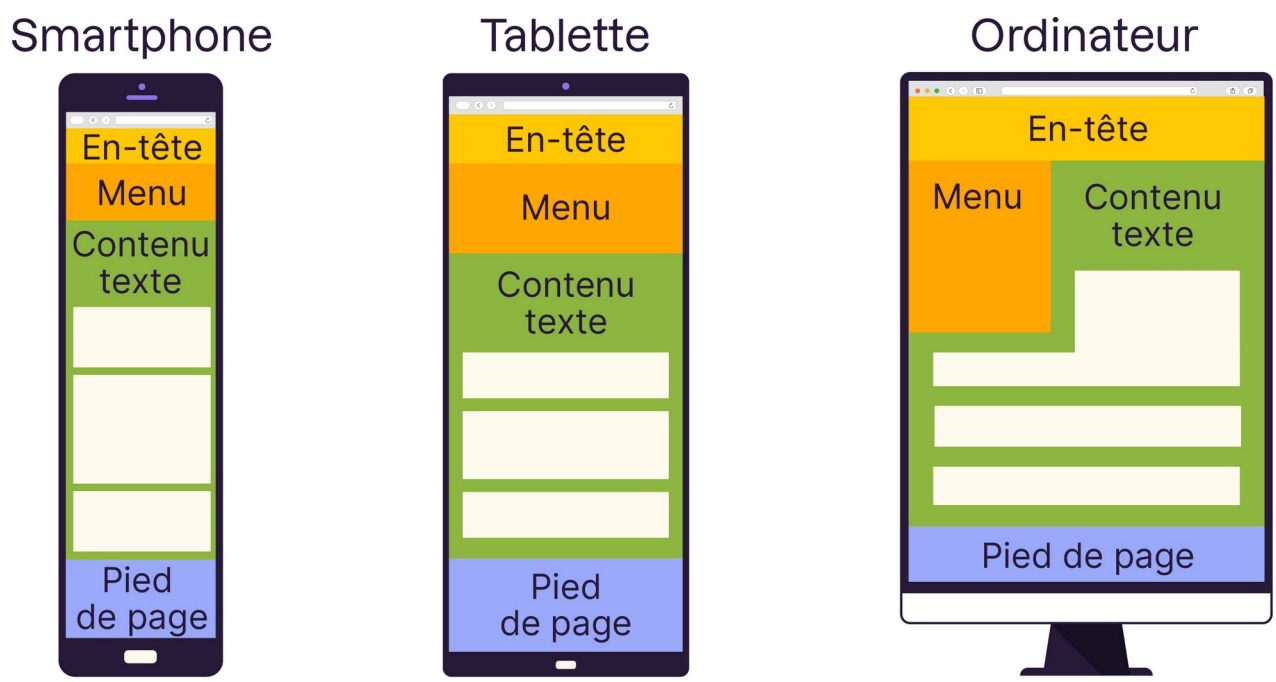
6.2. Quand faire du CSS responsive ?

Lorsque le site internet est destiné à toucher un maximum de personne car il doit être accessible sur le plus de supports possible.

6.3. Comment faire du CSS *responsive* ?

Pour faire du CSS responsive, il faut utiliser les ***media queries*** : il s'agit de règles qui indiquent quand on doit appliquer des propriétés CSS en fonction de la taille de l'écran sur lequel s'affiche le site WEB.

Ainsi, chaque support (smartphone, tablette, ordinateur, etc.) possède ses propres règles CSS et donc son propre agencement d'éléments.



Etape 1 : mettre à jour le <meta>

Ajoutez la balise <**meta**> suivante dans le <**head**> de votre document :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Etape 2 : construire une media query

Il existe énormément possibilités pour construire une media query.

Voici deux règles très utilisées et qui seront suffisantes pour le cours (la taille 1024px est à adapter selon les besoins) :

```
/* S'active lorsque la largeur de l'écran est inférieure à 1024px de large */
@media only screen and (max-width: 1024px){
}
```

```
/* S'active lorsque la largeur de l'écran dépasse 1024px de large */
@media only screen and (min-width: 1024px){
}
```

Etape 3 : ajouter des règles CSS à la media query

Il faut maintenant compléter le corps de la media query c'est-à-dire les règles CSS qui vont s'activer pour un certain type d'écran.

```css
/* Paragraphes en bleu par défaut */
p {
    color: blue;
}

/* Nouvelles règles si la fenêtre fait au plus 1024px de large */
@media only screen and (max-width: 1024px) {
    p {
        color: red;
        background-color: black;
        font-size: 1.2em;
    }
}
```

6.4. Sources

- https://dequeuniversity.com/library/responsive/1-non-responsive
- https://openclassrooms.com/fr/courses/1603881-creez-votre-site-web-avec-html5-et-css3/8061510-utilisez-le-responsive-design-avec-les-media-queries

7 Quelques ressources pour aller plus loin…

Utiliser plusieurs arrière-plans	
Concevoir des interfaces « *responsives* »	
Maîtriser la superposition des éléments	